Banco de Dados

Repositório Central de Concursos de TIC · UFRRJ & Transpetro 2026

Caderno Consolidado de Banco de Dados — teoria completa, mnemônicos e pegadinhas de prova.

Gerado automaticamente em 10/11/2026.

Sumário

• Banco de Dados T01 — Introdução e Modelos de Dados
• Banco de Dados T01 — Modelagem de Dados (DER)
• Banco de Dados T02 — Abordagem Relacional
• Banco de Dados T03 — Normalização
• Banco de Dados T06 — Modelagem Dimensional
• Banco de Dados T07-09 — SQL, DDL e DML
• Banco de Dados T10 — Propriedades ACID das Transações
• Banco de Dados T10 — SGBD: Arquitetura ANSI/SPARC
• Banco de Dados T11 — Banco de dados NoSQL

Banco de Dados T01 — Introdução e Modelos de Dados

Definições

Termo	Definição
	Banco de Dados (BD)
	SGBD
	SQL
	SQL Server

Modelos de Banco de Dados

Modelo	Descrição
	Conceitual
	Lógico
	Físico

• Tudo parte da **análise de requisitos** (é dela que sai o modelo conceitual).

• O **conceitual** é o de **alto nível** e oferece conceitos **próximos dos usuários**; é representado pelo modelo **Entidade-Relacionamento** (DER).

• O **relacional** é um dos modelos do lógico; já o **físico** é o projeto de baixo nível que o SGBD executa.

O modelo lógico é composto por:

• Modelo em **rede**

• Modelo **hierárquico**

• Modelo **relacional**

• Modelo **relacional a objetos** (objeto-relacional)

Etapas de um Projeto de Banco de Dados

```
Cenário → Engenharia de Requisitos → Modelo Conceitual → Modelo Lógico → Modelo Físico → BD acessado pelo SGBD
```

Data Lake e o perigo do Data Swamp

Termo	O que é
	Data Lake
	Data Swamp
	Data Silo
	Dark Data
	Data Mart
	Data Mesh

> ■■ **PEGADINHA**
>
> Banca monta a questão como "o que surge quando o lake cresce sem governança" → **Data Swamp**. Não confundir com *dark data* (guardado e esquecido) nem *silo* (isolamento entre áreas — o swamp é falta de curadoria no próprio lake).

Curadoria de dados (data stewards)

Na governança de dados, quem **documenta, mantém e garante qualidade** para uso analítico é o **curador de dados** (data steward) — papel **organizacional/de negócio**, não técnico:

Papel	O que faz	NÃO é responsável por
	Curador de dados	Documenta metadados, regras e definições; garante qualidade e prontidão analítica
	Cientista de dados	Analisa e modela

	Consumidor	Só usa os dados
	DBA	Infraestrutura técnica (backup, performance, segurança)

■ MEMORIZAR

"Quem **documenta e mantém** os dados prontos para análise" = **curador**. A palavra "analítico" no enunciado é anzol para **cientista de dados** — mas a atividade descrita é de curadoria, não de modelagem.

Banco de Dados T01 — Modelagem de Dados (DER)

Modelo Conceitual

- **Entidade**: conjunto de **objetos do mundo real** sobre os quais se quer manter informações no banco (Produto, Cliente, Carro, Contrato, Conta...) — representada por um **RETÂNGULO**
- **Atributo**: propriedade ou característica que **descreve** uma entidade — também chamado de **campo** (ex.: CARRO → Modelo, Marca, Cor, Categoria, Consumo, Valor)
- **Relacionamento**: conjunto de **associações de ocorrências** entre entidades (representado por um losango no DER)

Exemplo: `Empregado` —◊— `Departamento`

DER é a modelagem **conceitual** usada para representar entidades e relacionamentos de domínio.

■■ INFO

O modelo **Entidade-Relacionamento** foi criado por **Peter Chen**. É a representação gráfica do modelo **conceitual** — alto nível, conceitos próximos dos usuários — e é a **primeira** etapa de modelagem depois da análise de requisitos.

Cardinalidade

Tipo	Descrição
	1:1 (um para um)
	1:n (um para muitos)
	n:n (muitos para muitos)

Consequências práticas

- **1:1** → em regra, **fusão das tabelas** numa única relação (também dá para deixar a FK de um dos lados com restrição de unicidade).
- **1:n** → a **FK vai para o lado N**.

- **n:n** → é criada uma **tabela intermediária** (associativa).
- **NÃO confundir 1:N com N:N.**

Notação de cardinalidade mínima

- Mínima **0** → participação **opcional**
- Mínima **1** → participação **obrigatória**
- Máxima **1** → 1 ou N (0 não é)

Chaves

- **Chave artificial sem significado** → **chave surrogada**
- Ligações entre fatos e dimensões são feitas por **chaves estrangeiras** no contexto relacional/dimensional.

Especialização/Generalização — totalidade × disjunção

Quando uma **superclasse** se fatia em **subclasses** (o clássico: PESSOA → PROFESSOR, ALUNO, FUNCIONARIO_ADMINISTRATIVO), o DER impõe **duas restrições independentes**. A pegadinha é que a banca conta com você decidir as duas de uma vez só, quando são duas perguntas diferentes:

Restrição	Pergunta de bolso	Valores
	Totalidade	A instância da superclasse **precisa** cair em alguma subclasse?
	Disjunção × sobreposição	A instância pode assumir **mais de um** papel ao mesmo tempo?

Na questão da universidade: a regra 1 ("toda PESSOA deve ser, pelo menos, um ALUNO, ou um PROFESSOR, ou um FUNCIONARIO_ADMINISTRATIVO") resolve a **totalidade** — é total. A regra 2 ("pode ser ALUNO e PROFESSOR ao mesmo tempo, mas não pode ser PROFESSOR e FUNCIONARIO_ADMINISTRATIVO") resolve a disjunção — e ela **não é uniforme**: um par é permitido, outro é proibido.

> **■■ PEGADINHA**
>
> Pela leitura clássica, bastaria haver **um** par simultâneo permitido para classificar tudo como **sobreposta** (a alternativa "Total e Sobreposta" estava lá esperando). A FGV, porém, aceitou **"Total e Mista (parcialmente Disjunta e parcialmente Sobreposta)"** — rótulo que a doutrina formal nem sempre usa. Lição: quando o enunciado traz **pares permitidos e pares proibidos** ao mesmo tempo e a banca oferece a categoria "mista", é ela que está sendo testada — a alternativa *mais correta* ganha da alternativa *mais elegante*.

> **■ DICA**
>
> Decisão em dois passos, nessa ordem: **(1)** olhe só a regra de completude → total ou parcial; **(2)** olhe só a simultaneidade → disjunta, sobreposta ou **mista** (par permitido **e** par proibido convivendo). Nunca decida as duas restrições com a mesma frase.

Entidade forte × fraca e o relacionamento identificador

Pensa em duas empresas: uma que **se banca sozinha** e outra que **só existe por causa do contrato com a primeira**.

- **Entidade forte**: possui atributo ou conjunto de atributos capaz de identificá-la unicamente — a chave primária é **dela**.
- **Entidade fraca**: não possui. A propriedade identificadora nasce da combinação da **chave parcial** (atributo discriminador) com a **chave primária de outra entidade**.

O vínculo entre as duas pode ser de dois jeitos:

Relacionamento	O que acontece	Exemplo
	Identificador	A PK da entidade-pai **entra na PK** da entidade-filha — a filha depende do pai para **existir**
	Não identificador	A filha **não depende em sua totalidade** do pai: cada uma tem identificador próprio, e a FK só referencia

■ CONEXAO

Na notação **IDEF1X** da modelagem relacional, o relacionamento identificador é representado por uma **linha sólida**, enquanto o relacionamento não identificador é representado por uma **linha tracejada**. Somando o conceito (quem depende de quem para existir) ao traço visual da notação, você resolve qualquer questão de diagrama.

Banco de Dados T02 — Abordagem Relacional

■■ INFO

Abordagem Relacional e Integridade Referencial: estudo completo de chaves (superchave, chave candidata, primária, alternativa e estrangeira), restrições de integridade (domínio, entidade, referencial e transacional), dependências funcionais e cálculo de fecho de atributos.

Do que se trata?

Pensa na planilha mais arrumada que você já viu: cada linha é uma ocorrência, cada coluna é uma característica, e nenhuma célula guarda "duas coisas" ao mesmo tempo. Essa é a essência do **modelo relacional** — os dados vivem em **relações** (tabelas) com regras rígidas de integridade, e não ficam soltos no arquivo.

Vocabulário que cai em prova

A FGV adora cobrar a nomenclatura: o mesmo objeto tem três nomes, e a banca mistura tudo na alternativa.

Termo	Sinônimos

	Relação
	Tupla
	Atributo
	Domínio
	Cardinalidade
	Superchave
	Chave candidata
	Chave primária (PK)
	Chave estrangeira (FK)

■ **DICA**

Tudo que a dica de prova precisa: **PK** é a "escolhida oficialmente"; **candidata** é a mínima; **superchave** não precisa ser mínima. "FK aponta pra quem manda na outra tabela."

Chave primária x chave estrangeira

A pegadinha clássica conta com você mantendo dados repetidos nas tabelas. A saída é sempre a mesma receita de normalização:

- **Chave primária** — identifica a linha **dela mesma** (não nula, única).
- **Chave estrangeira** — carrega a identidade de **outra** relação, criando o vínculo.

Dependência funcional — o passo a passo de prova

Antes de qualquer forma normal, você precisa saber **ler uma dependência funcional numa tabela**: uma coluna _C_2 depende funcionalmente de _C_1 (escrito _C_1 → _C_2) quando, **para qualquer linha da tabela, a cada valor de _C_1 aparece sempre o mesmo valor de _C_2**. Pensa no garçom: mesma mesa, mesmo pedido anotado — se uma vez a mesma mesa aparecer com dois pedidos diferentes no bloco, a "dependência" quebrou.

Passo a passo para resolver a questão analisando a tabela:

■ Olhe para a alternativa (A), que indica $A \rightarrow B$.

■ Verifique os valores da coluna A: se o valor "1" na coluna A estiver associado ao valor "2" na coluna B em uma determinada linha, confira se **todas as vezes** que o "1" aparecer na coluna A, o "2" também aparecerá na coluna B.

■ Se você encontrar uma linha com A="1" e B="2", e outra linha diferente com A="1" e B="3", a dependência funcional foi quebrada — logo, a afirmação $A \rightarrow B$ é falsa. Aplique essa **varredura visual** em todas as alternativas até encontrar aquela em que a regra se mantém intacta para todas as instâncias da tabela.

Nas instâncias dessa questão: **A** → **B** é verdadeira (T1: A=1 → B=3; A=2 → B=2; A=4 → B=2 — cada A determina um único B); **A** → **C** é falsa (T3: A=1 → C=100 e C=110 — o mesmo A determina dois C); **B** → **A** é falsa (T1: B=2 → A=2 e A=4); **C** → **A** é falsa (T3: C=100 → A=1 e A=2); **E** → **C** é falsa (T3: E=12 → C=100 e C=110). Resumindo: **cada valor de X existe apenas um valor correspondente de Y.** (T1 e T3 referenciam as tabelas exibidas na imagem do enunciado.)

Dica de prova — varredura em três tempos: (1) pega a alternativa; **(2)** procura na coluna do determinante um valor repetido; **(3)** vê se o correspondente mudou. Mudou → quebrou → falsa. E repare nas alternativas da banca: duas das falsas são a **seta invertida** (B → A e C → A) — a FGV testa se você lê a dependência da esquerda para a direita.

Chaves candidatas — o fecho na mão

Achar chave candidata no calor da prova tem um macete: **observe quais atributos não aparecem no lado direito de nenhuma dependência funcional** — esses obrigatoriamente têm de entrar em qualquer chave candidata. A partir daí, é só "fechar" conjuntos de atributos até achar quem determina tudo.

Exemplo com F = {D → A, A → B, BC → DE} e a relação R(A, B, C, D, E): o **C** não apareceu no lado direito de nada, então toda candidata precisa dele. Teste o fecho ($) das combinações:

Teste	Fecho	Resultado
	$C	{C} — nenhuma dependência tem só C à esquerda
	$AC	{A,C} → A → B → {A,B,C} → BC → DE → **{A,B,C,D,E}**
	$DC	{D,C} → D → A → {A,C,D} → A → B → {A,B,C,D} → BC → DE → **{A,B,C,D,E}**
	$BC	{B,C} → BC → DE → {B,C,D,E} → D → A → **{A,B,C,D,E}**

Detalhe fino: a alternativa oficial (C) cita **{AC} e {DC}**, mas o fecho mostra que **{BC} também é minimal e determina tudo** — o gabarito comentado registra a mesma conta (BC+ = ABCDE). Não encerre a busca na primeira chave que aparecer: teste as combinações mínimas, é para isso que serve o fecho.

Amarração com as Formas Normais (Normalização): a relação está na **3FN** e **não** na BCNF, porque em A → B e D → A os determinantes A e D **não são superchaves** — na BCNF todo determinante tem de ser superchave; na 3FN basta o determinante ser superchave **ou** o dependente ser atributo primo.

Entidade forte x entidade fraca

- **Forte**: tem atributos próprios capazes de identificá-la (a PK é dela).
- **Fraca**: não tem; a identificação nasce da combinação da **chave parcial** (discriminador) com a **PK de outra entidade**.

Notação IDEF1X: linha sólida × tracejada

Na notação IDEF1X o **relacionamento identificador** (a FK entra na PK da entidade filha — filha fraca) é desenhado com **linha sólida**; o **não identificador** (a FK não participa da PK — filha forte) usa **linha tracejada**. É a "pegada" visual de forte × fraca.

> ■ **DICA**
>
> Sólido = "vida presa ao pai" (identificador); tracejado = "guarda a chave do pai só de referência" (não identificador). Se a banca mostrar o desenho em vez do texto, é esse par que ela quer.

Integridade referencial

Quem integra a prática com o conceito: a FK **só aceita valores que existam na PK referenciada** (ou NULL). Foi isso que a banca cobrou na modelagem das fiscalizações:

- `MUNICIPIO` (PK) ← `ORGAO` tem FK para `MUNICIPIO` (cada órgão pertence a um único município).
- `FISCALIZACAO` guarda **apenas** `id_orgao` e `id_auditor` como FKs — o município é alcançado **indiretamente** pelo órgão. Menos redundância, mesma consulta.

O guarda-roupa das integridades

São **cinco** e a banca vive embaralhando os nomes:

Integridade	Cuida de...
	Entidade
	Referencial
	Domínio
	Semântica
	Transacional

> **■■ PEGADINHA**
>
> "Campo que só aceita número" é **domínio**; "chave única não nula" é **entidade**. A troca dessas duas é o blefe preferido da FGV. E se o enunciado falar em "valor que deve existir como chave em outra tabela", é **referencial** — não entidade, não domínio. Integridade **transacional** é o nome bonito do ACID — quem vê "estados de transação" deve pensar em commit/rollback, não em FK.

> **■ CONEXAO**
>
> Essa "pergunta pelo outro lado" é o mesmo raciocínio do **star schema** (T06): tabela associativa / fato guarda as **chaves**, e você junta por joins. Se um dia a mesa ficar quente, procure sempre o **identificador**, nunca o texto do nome repetido.

Onde eu erraria e por quê?

Eu tenderia a marcar "restrição de entidade" quando o campo aceitasse só número — afinal "o tipo está restringido"... Mas restrição de tipo é **domínio**. A palavra "entidade" só aparece quando falamos da **identificação da tupla** (PK não nula). Guarde: *tipo → domínio; chave → entidade*. Outra que pega todo mundo: **inverter forte × fraca** e chamar de "forte" a entidade que usa a chave alheia (4ª assertiva) — a fraca é a que não se identifica sozinha; no relacionamento identificador a PK do pai entra na PK da filha (linha sólida no IDEF1X), enquanto no não identificador ela entra apenas como FK comum (linha tracejada). E na varredura de dependência funcional eu pararia na primeira alternativa que "parecesse certa" — a receita é varrer **todas**, tupla por tupla, porque a falsa costuma ser a mesma regra com a seta ao contrário.

Como memorizar sem decorar?

> **■ MEMORIZAR**

"Relação é a família, a linha é o filho, a coluna é o traço." A mesa (tabela) chamada relação, cada filho (registro) único na PK, e o traço (atributo) herdado do domínio. Entidade **forte** paga a própria conta; **fraca** depende do CNPJ do pai (PK alheia + senha parcial).

O que revisar e quando?

- A **terminologia relação/tupla/atributo** e a **classificação forte × fraca** são cobradas quase todos os anos em questões de V/F (FGV 2026/TJ-SC).
- Reforce a **integridade referencial** junto com modelagem e com SQL de joins (T07).
- **Varredura de dependência funcional** e **fecho de atributos para achar chave candidata** são os dois procedimentos de "mão na massa" do tópico — a FGV tem cobrado o cálculo, não só a definição.

Lição para outras matérias

O mesmo padrão do "nome repetido que gera inconsistência" aparece em **Raciocínio Lógico** (deduplicação de premissas), **Governança** (mestre-dados/qualidade da informação) e **Infra** (índices e estatísticas de custo). Em qualquer matéria: *redundância mal planejada = inconsistência; centralização em tabela de referência = sanidade.*

Banco de Dados T03 — Normalização

■■ INFO

Normalização das Estruturas de Dados: estudo progressivo da 1ª Forma Normal (1FN — atributos atômicos e sem grupos repetitivos), 2ª Forma Normal (2FN — eliminação de dependências funcionais parciais), 3ª Forma Normal (3FN — eliminação de dependências funcionais transitivas) e Forma Normal de Boyce-Codd (BCNF).

Do que se trata?

Normalização é a disciplina de **organizar as prateleiras da despensa relacional**: cada informação é guardada **uma única vez**, no lugar certo. O objetivo não é "deixar bonito" — é **matar as anomalias** que nascem do mau planejamento de tabelas.

Por que cai em prova?

A FGV cobra normalização de três jeitos: (a) **definição pura** de cada forma normal, (b) **análise de dependência funcional** (qual cadeia viola qual FN) e (c) **identificar o que cada FN elimina**. É o tópico com mais questões FGV recentes entre as 20 de Banco de Dados: 1FN, 2FN e 3FN aparecem literalmente.

As três formas normais que dominam a prova

Forma	Elimina	Exigência

	1FN	Grupos repetitivos / valores não atômicos
	2FN	**Dependência parcial** (de parte da chave composta)
	3FN	**Dependência transitiva**

Os exemplos que a banca costuma desenhar:

- Endereço guardado numa **única string** (rua + número + CEP) → campo composto, não atômico → viola a **1FN**.
- `Telefone1`, `Telefone2`, `Telefone3` na mesma tabela → **grupo repetitivo** (coluna repetida para o mesmo tipo de dado) → viola a **1FN**.
- Vetor/lista dentro de uma célula → mesma coisa, viola a **1FN**.

> ■ **DICA**
>
> Resumo da despensa: **1FN = sem caixa com dois itens**; **2FN = sem filho que só dependa de metade do pai (chave composta)**; **3FN = sem sobrinho mandando no primo (não-chave → não-chave)**.

As pegadinhas clássicas

> ■■ **PEGADINHA**
>
> **"Chave simples na 1FN já está na 2FN"** — verdadeiro: sem chave composta não há dependência parcial para eliminar.
>
> **"3FN não cuida de chave composta"** — depende: o alvo da 3FN é a dependência **transitiva**, não a parcial. A 2FN é que cuida de chave composta.
>
> **Índice único, chave estrangeira e primária NÃO definem forma normal** — elemento estrutural confunde o candidato (/C).

A cadeia que a FGV ama: dependência transitiva

Obrigatório reconhecer o padrão `PK` → `não-chave X` → `não-chave Y`:

- Enunciado FGV 2026/NAV Brasil: `ID_Veiculo (PK)` → `ID_Fabricante` e `ID_Fabricante` → `Nome_Fabricante`.
- `Nome_Fabricante` depende da chave **por intermédio** de outro atributo não-chave → **violação da 3FN**.
- Solução: tirar `Nome_Fabricante` de `TB_VEICULO`, deixar só `ID_Fabricante` (FK) e criar `TB_FABRICANTE (ID_Fabricante PK, Nome_Fabricante)`.

Definição formal (a que a banca parafraseia): uma tabela está na **3FN** quando está na **2FN** e **nenhum atributo não-chave depende transitivamente da chave primária**. Dito ao contrário: todo atributo não-chave deve depender **diretamente** da chave, e **de mais nada**.

O contraponto que a FGV monta: `ID_PROJETO (PK)` → `ID_DEPARTAMENTO` → `NOME_DEPARTAMENTO`. Como `NOME_DEPARTAMENTO` depende de `ID_DEPARTAMENTO`, que **não é chave primária nem candidata**, temos dependência transitiva → **3FN**. E se a chave fosse composta por `(ID_PROJETO, ID_DEPARTAMENTO)`? Aí seria **2FN**: `ID_PROJETO` → `NOME_PROJETO` e `ID_DEPARTAMENTO` → `NOME_DEPARTAMENTO` seriam duas dependências parciais (atributos não-chave dependendo de só metade da chave).

Pegadinha de banca: repare no aviso do enunciado: "**ID_DEPARTAMENTO não é chave primária**". É a banca te tirando da 2FN — dependência parcial **só existe com chave composta**, e essa relação tem PK simples (só `ID_PROJETO`). Quem lê "um atributo depende do outro" e marca 2FN sem checar a PK cai direito.

> ■ **RESUMO**
>
> Nos aquecer pro padrão: **dependência parcial** → **2FN**; **dependência transitiva** → **3FN**; **multivalorado/atômico** → **1FN**; **BCNF** é só a versão mais rígida da 3FN para chaves candidatas sobrepostas — distrator clássico.

Anomalias: o motivo da existência

Normalização existe para **extinguir**:

- **Anomalia de inserção** — não consigo gravar a informação sem gravar a vizinha.
- **Anomalia de exclusão** — excluir um registro leva junto um dado que eu queria manter.
- **Anomalia de atualização** — corrigir um valor exige alterar vários lugares (risco de divergência de grafia, como no).

Caso de prova: tabela `PROJETOS_AUTORES` com PK composta `(Título_PL, Cod_Autor)` e o atributo `Nome_Autor` dependendo só de `Cod_Autor`, ou seja, de **parte da chave** — dependência parcial, então a tabela nem chega à 2FN (está na 1FN). Se a tarefa for alterar o `Nome_Autor` de um autor específico, várias linhas mudam de uma vez, sem inserir nem remover nada: **anomalia de atualização**. Não é inserção (nada entrou) nem exclusão (nada saiu) — é "correção espalhada", o defeito clássico da redundância sem controle.

Tem discordância nos comentários da questão: um colega argumenta que, **segundo Codd**, a tabela nem estaria na 1FN, porque toda linha precisa ser única e identificável por uma chave — e a instância teria duas tuplas com a mesma combinação de `(Título_PL, Cod_Autor)`. O gabarito oficial, contudo, é **1FN + anomalia de atualização** — na dúvida, é esse que vale.

> ■ **CONEXAO**
>
> Em **Redes** você viu o mesmo dilema lógico: dados duplicados em vários roteadores geram loop de inconsistência; centralizar a "fonte da verdade" (SPF/IGP) é análogo a normalizar a tabela. A redundância com controle é ok; redundância espalhada é bug.

Onde eu erraria e por quê?

Eu marcaria "índice único sobre a chave primária" como se fosse 1FN — porque soa técnico e "de banco". Mas 1FN não fala de implementação física: é sobre a **célula** (atómica) e sobre **caixas repetidas** na mesma coluna. Se a alternativa menciona índice, PK, FK — é distrator. Outro tropeço: ver "endereço" ou "telefone1/telefone2" na achar que é caso de chave composta, quando o problema é **atômico/grupo repetitivo** (1FN).

Como memorizar sem decorar?

> ■ **MEMORIZAR**
>
> **P. A. T.** — **P**arcial (2FN) → **A**tômico (1FN) → **T**ransitiva (3FN). E a regra de bolso: *1FN não cuida de chave, 3FN não cuida de chave composta.* Chave compostas são território da 2FN. Régua da normalização: **1FN** = só valores atômicos, sem grupos repetitivos; **2FN** = depende de **TODA** a chave;

3FN = depende **APENAS** da chave.

■ **DICA**

A Escada de Codd em uma frase por degrau:

1FN → pergunta-chave: "o valor da célula é atômico e único?" Se falar em índice, PK ou FK, é distrator — 1FN é conceito lógico, não físico.

2FN → pergunta-chave: "a chave primária é **composta**?" Se for simples, a tabela já está automaticamente na 2FN.

3FN → pergunta-chave: "algum não-chave depende de **outro** não-chave?" Se sim, é dependência transitiva → viola 3FN.

BCNF → é a 3FN mais rigorosa: todo determinante deve ser chave candidata. Distrator clássico da FGV — a questão descreve 3FN simples e a alternativa errada oferece "BCNF" como resposta.

■ **DICA**

4FN e 5FN raramente caem como gabarito, mas aparecem como distratores. Lembrete rápido: **4FN** elimina **dependências multivaloradas** (quando um atributo determina *múltiplos* valores independentes de outro); **5FN** elimina **dependências de junção**. Se o enunciado não citar explicitamente esses termos, a resposta quase certamente é 1FN/2FN/3FN.

■ **ALERTA**

Chaves candidatas são todas as colunas (ou combinações) que poderiam ser chave primária. Para calcular: com *n* atributos e *k* dependências funcionais, verifique quais combinações determinam todos os outros. A banca gosta de dar um conjunto de DFs e pedir qual chave candidata é válida — teste cada alternativa: ela determina **todos** os outros atributos diretamente?

O que revisar e quando?

A sequência 1FN→2FN→3FN cai em **quase toda prova de banco relacional** (FGV 2026: NAV Brasil, MPE-ES, TJ-SC, TCE-PE, AL-RO; FGV 2022: Prefeitura de Manaus). Revise sempre os casos clássicos de dependências parciais e transitivas antes da prova. A tendência recente é cobrar a **dependência funcional concreta**, não só a definição decorada.

Alerta de banca: em a FGV publicou **gabarito preliminar C** (BCNF) e o **oficial passou para E** (dependência transitiva). Lição: escolha pela **descrição da dependência**, não pelo rótulo da forma normal citado na alternativa.

A **desnormalização** (o outro lado da moeda, com trade-off de leitura × integridade) é o pilar da **Modelagem Dimensional (Data Warehouse)**.

Lição para outras matérias

O mesmo *refactoring* aparece em **Raciocínio Lógico**: quando duas premissas têm a "mesma informação" em duas condições diferentes, você **extrai a relação transitiva** (como a 3FN extrai a tabela). Em **Governança**, é o prato do dia do gerenciamento de qualidade de dados: *single source of truth* é normalização com outro nome.

Banco de Dados T06 — Modelagem Dimensional

■■ INFO

Modelagem Dimensional (Data Warehouse / OLAP): arquitetura de tabelas Fato e Dimensão, comparação entre Esquema Estrela (*Star Schema* — dimensões desnormalizadas) e Esquema Floco de Neve (*Snowflake Schema* — dimensões normalizadas), e operações analíticas de filtragem e agregação.

Do que se trata?

A modelagem dimensional é o "mapa do Data Warehouse": uma **tabela de fatos** no centro (o que aconteceu — vendas, fiscalizações, vínculos) rodeada de **tabelas de dimensões** (quem, onde, quando, o quê). Nada a ver com banco transacional supernormalizado: aqui a régua é **leitura rápida**, não economia de armazenamento.

Star Schema (Esquema Estrela) — puro e clássico

- **Dimensões desnormalizadas**, ligadas **diretamente à tabela fato**, em **um único nível lógico plano**.
- A ligação é feita por **chaves estrangeiras na tabela fato** que referenciam as chaves das dimensões — nada de hierarquia externa.
- Foco em **otimização de consultas e leitura de informações**: como não há sub-tabela para descer, o **número de JOINs cai** e a leitura fica curta e previsível.

2.1 Por que o esquema em estrela existe: as três operações

O motivo de o estrela ser o padrão em data warehouse é que ele foi desenhado para **exatamente três operações analíticas** — e a banca vai descrever as três:

Operação	O que faz	Onde aparece no SQL
	Filtrar (*slice*)	Restringe o recorte — "só vendas de 2025"
	Agrupar (*dice*)	Escolhe por qual eixo fatiar — "por região"
	Agregar (*measure*)	Calcula o número — `SUM`, `AVG`, `COUNT`

Estrutura do modelo: a **tabela fato** guarda as **métricas/medidas** numéricas (vendas, receita, quantidade); as **tabelas de dimensão** guardam os **atributos descritivos** que dão contexto ao fato (**tempo**, **produto**, **cliente**, **localização**).

■ DICA

Guarde o tripé com as palavras do enunciado: *filtrar* = `WHERE`, *agrupar* = `GROUP BY`, *agregar* = função de agregação. Se a questão afirmar que **os atributos das tabelas fato e dimensão** podem ser usados para as três coisas, está **certo** — porque a dimensão é justamente o eixo que dá sentido aos números. Se disser que só a *tabela fato* filtra, ou que a dimensão *armazena* as medidas, está errado: quem guarda medida é a **fato**.

■■ PEGADINHA

A inversão clássica é trocar os papéis: **fato = medida, dimensão = atributo**. A banca escreve "atributos das tabelas fato e dimensão podem ser usados para filtrar, agrupar e agregar os fatos" e quem lê rápido entende "a medida filtra" e marca errado. A medida **é o que se agrega**; o atributo da dimensão **é o que filtra e agrupa**.

Snowflake Schema (Floco de Neve)

- Dimensões **altamente normalizadas**, quebradas em **ramificações hierárquicas em cadeia**.
- Ligação entre **fatos e dimensões** por **chaves estrangeiras** no contexto relacional/dimensional.
- Por ser mais normalizado, tende a exigir **mais operações de JOIN** nas consultas.
- Custo: **maior consumo de memória e menor desempenho** de leitura.

■■ PEGADINHA

A FGV descreve floco de neve e pergunta "qual a característica do **estrela puro**?": dimensões **desnormalizadas e planas**, ligadas direto na fato. Quando a alternativa falar em "forte normalização + ramificações em cadeia", é **snowflake** — distrator clássico.

Fatos e granularidade

A **tabela de fatos** guarda as **chaves das dimensões** + medidas numéricas. A **granularidade** (= o que cada linha da fato representa) é o ponto fino que decide a resposta certa:

- `GROUP BY ID_SERVIDOR HAVING COUNT(DISTINCT ID_ORGAO) >= 2` → cada linha = um **servidor** em órgãos distintos (acumulação de cargos).
- Inverter para `GROUP BY ID_ORGAO` → cada linha = um **órgão** com ≥ 2 servidores.

■ ALERTA

Mesmo SQL, granularidades opostas = respostas opostas. A dimensão "quem" na linha manda na interpretação (gancho com, em SQL/DDL/DML).

Desnormalização — o pacote de benefício com a conta na mão

A **desnormalização** é a introdução **intencional** de redundância para atingir um objetivo de desempenho. Em DW/OLAP ela serve para:

- **Otimizar tempo de resposta** em consultas complexas e relatórios.
- **Simplificar o SQL**: menos *joins* entre tabelas = menos processamento.
- **Facilitar o uso pelo negócio** — é o pilar da modelagem dimensional (Kimball), com tabelas fato e dimensão.
- Em **NoSQL/documentos**, reduz dependências: o registro vira um bloco único, sem consulta interna e externa.

O trade-off é sempre o mesmo pacote:

Ganha	Paga
	Leitura/consulta analítica mais rápida (menos JOINs)

	SQL mais simples para o analista

• A **desnormalização não serve principalmente para escrita** — no DW, o foco é leitura; ambiente desnormalizado costuma ser **pior** para *inserts/updates*.

• A **3FN é ótima para OLTP** (integridade, escrita) e **não é a escolha ideal para DW**, que exigiria muitos *joins* nas agregações.

• **Não é exclusiva de NoSQL** nem inviável em SQL — é prática consolidada em SGBDRs (Oracle, SQL Server, PostgreSQL).

• Dimensão **plana e consolidada = star schema**; normalizar dimensão = **snowflake**.

■ ALERTA

As desvantagens que a banca cobra como "consequências da desnormalização": **redundância e desperdício de espaço**, **risco de inconsistência** (atualização falha em um dos lugares e some a "fonte única da verdade"), **complexidade de manutenção/governança** (ilhas de informação — *data silos*), **piora do CRUD** e até **impacto em Machine Learning** (dado redundante é dado ruim — *"garbage in, garbage out"*). Efeito colateral de projeto bem intencional, não bug.

■ DICA

Gatilho para questões de OLTP × OLAP:

Cenário **OLTP**: 3FN, integridade, poucos JOINs de escrita → prioridade é a **corretude** dos dados.

Cenário **DW/BI**: desnormalização, star schema, joins reduzidos → prioridade é a **velocidade de leitura**.

Se a alternativa disser que a desnormalização otimiza **escritas** (inserts/updates), está **errada** — o benefício é na **leitura analítica**.

Dimensão plana/consolidada = star schema; dimensão normalizada em camadas = snowflake. Quando a questão descrever "dimensão única, plana, sem relacionamentos hierárquicos externos", é **estrela** — não floco.

■ CONEXAO

É o **mesmo dilema OLTP × DW** que a FGV gosta de inverter: no transacional, normalização protege integridade; no analítico, desnormalização protege a leitura. E é o mesmo trade-off de **Infraestrutura**: disco barato + leitura veloz (estrela desnormalizada) versus espaço/fidelidade (floco normalizado). Abordagem Inmon (consolidação normalizada, mais lenta no início) × Kimball (dimensional, mais rápida no início) é o "quem paga antes, quem paga depois". O *single source of truth* (T03) vive só no **floco**; no estrela, repete-se sem medo.

Como memorizar sem decorar?

■ MEMORIZAR

Estrela = pizza direto no prato (uma fatia, sem etapas); **floco = bolo em camadas** (cada camada é uma subdimensão normalizada). Fato = o centro; dimensões = a borda; granularidade = o tamanho da fatia.

O que revisar e quando?

• **Fato × dimensão e as três operações (filtrar/agrupar/agregar)** — o enunciado "certo ou errado" do esquema em estrela é a forma mais provável de o tema cair; basta saber quem guarda medida e quem

guarda atributo.

• **Dimensões × fatos** e a distinção **estrela × floco** aparecem com frequência em provas de DW da FGV (2026/NAV Brasil).

• **Desnormalização em DW** é a pergunta "qual o objetivo e qual o custo?" — resposta: menos *joins*, mais leitura, mais espaço e mais risco de anomalia.

• Revise **junto com normalização (T03)** — a banca adora contrapor "desnormalizado no DW" com "normalizado no transacional".

• **Conexão com SQL (DDL/DML/DQL)**: muitas questões de prova citam modelagem dimensional apenas como cenário introdutório do enunciado, mas cobram na prática a classificação dos comandos SQL (`CREATE`, `ALTER`, `SELECT`, `INSERT`). Não confunda o cenário do texto com o comando exigido.

Banco de Dados T07-09 — SQL, DDL e DML

■■ INFO

Linguagem SQL (DDL, DML, DQL, DCL e DTL): classificação dos subconjuntos da linguagem SQL, criação e alteração de estruturas (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`), manipulação e consulta de dados (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `JOIN`, `GROUP BY`, `HAVING`), objetos de banco (`VIEW`, `PROCEDURE`, `TRIGGER`) e otimização por índices.

Do que se trata?

SQL é a linguagem universal de conversa com o banco. A pegadinha da vida real é que **uma única linguagem "fala quatro dialetos"**: um dialeto define a estrutura (DDL), outro mexe no conteúdo (DML), outro controla acesso (DCL) e outro gerencia transação (DTL). A prova cobra exatamente *"qual comando pertence a qual dialeto?"*.

Os subgrupos — o mapa que nunca pode sair da cabeça

Subgrupo	O que faz	Comandos
	DDL (Definition)	Estrutura / metadados
	DML (Manipulation)	Conteúdo
	DQL (Query)	Consulta
	DCL (Control)	Permissões
	DTL (Transaction)	Transações

■ DICA

DDL explica o que é o quarto; **DML** faz a mudança; **DCL** decide quem tem a chave; **DTL** garante "ou fecha tudo, ou não fecha nada". cobrou o trio mais famoso: `CREATE TABLE` → `DDL`, `INSERT INTO` → `DML`, `GRANT` → `DCL`.

■■ PEGADINHA

> `COMMIT` e `ROLLBACK` **não** são DDL — são **DTL**. A banca já vendeu "comandos DDL do tipo COMMIT/ROLLBACK" como distrator. E o `SELECT`, se a prova tiver DQL, vai **no DQL**; se não tiver, coloque no DML sem pestanejar.

2.1 A matriz definitiva de classificação SQL

A maior parte das questões de SQL em provas pede para classificar os comandos em seus subconjuntos (DDL, DML, DQL, DCL ou DTL). Você não precisa decorar dezenas de casos isolados — precisa apenas de **uma pergunta de desempate** e de **uma matriz**:

> *Esse comando muda o **conteúdo** da tabela, muda a **estrutura** dela, muda **quem pode acessar**, ou controla a **transação**?*

Comando	Família	Agente sobre o quê?	Questão que cobra
	`CREATE TABLE` / `CREATE INDEX`	**DDL**	estrutura
	`ALTER TABLE` / `ALTER DATABASE`	**DDL**	estrutura
	`DROP TABLE` / `DROP INDEX`	**DDL**	estrutura
	`TRUNCATE TABLE`	**DDL**	conteúdo (mas sem `WHERE`)
	`INSERT`	**DML**	conteúdo
	`UPDATE`	**DML**	conteúdo
	`DELETE FROM`	**DML**	conteúdo
	`SELECT`	**DQL** (ou DML)	nada — só lê
	`GRANT` / `REVOKE`	**DCL**	permissões
	`COMMIT` / `ROLLBACK`	**DTL** (ou TCL)	transação
	`SAVEPOINT` / `ROLLBACK TO`	**DTL**	transação

O teste mais limpo é o da **tríade em sequência**: criar tabela → inserir registro → conceder permissão é, respectivamente, **DDL, DML, DCL**. Repare que essa questão também funciona como teste de "o que **não** é": `GRANT SELECT` é DCL, **não** DQL — ler dado e *dar* permissão de leitura são coisas diferentes.

> **■ MEMORIZAR**
>
> **Estrutura = DDL · Conteúdo = DML · Permissão = DCL · Transação = DTL.** Quatro palavras, quatro famílias. Se você só lembrar dessa linha, já mata e.

> **■ ALERTA**
>
> `ALTER TABLE` é sempre DDL, mas **executar sem erro** tem um segundo filtro: a alteração pode impor **nova** exigência sobre dados já gravados, e aí ela falha. O par que resolve é: **remover** restrição é seguro (elimina uma exigência), **adicionar** `UNIQUE` a coluna com valores repetidos **quebra** (viola a restrição no momento). Esse raciocínio — *afrouxar sempre pode; apertar exige dado compatível* — vale para qualquer `ALTER`, não só para esse item.

2.2 As quatro viagens da banca (e como não cair)

O edital não diz qual taxonomia usar, então cada banca adota a sua. A prova real é: **a alternativa certa é sempre a que segue a convenção que o enunciado ou a banca usaram** — e a grande maioria adota "SELECT é DQL, DML é INSERT/UPDATE/DELETE".

Viagem 1 — `SELECT` é DML ou DQL?

Autores antigos (Elmasri & Navathe) e a maioria das bancas regionais tratam `SELECT` como DML. Bancas que inventaram a categoria DQL o isolam. Em e a resposta veio **separando** os dois: "SELECT pertence à DQL" foi aceito, e a alternativa que misturou `SELECT` com `CREATE` foi derrubada. Ou seja: **separar nunca é erro; juntar com DDL é sempre erro.**

Viagem 2 — `COMMIT` é DML?

Nunca. É controle de transação (DTL/TCL). e existiram só para matar quem generalizou "comandos SQL = DML".

Viagem 3 — `TRUNCATE` é DML?

Não, é DDL — apaga o conteúdo mas é comando de definição, e é por isso que não aceita `WHERE`. Confundir "apaga dados" com "é DML" é o erro que a banca mais gosta.

Viagem 4 — `DENY` é DCL?

`DENY` é do SQL Server (T-SQL) e **não faz parte do SQL padrão**. Quando a banca lista `GRANT`, `REVOKE` e `DENY` como trio DCL, ela está descrevendo o SQL Server — mas em item que fala "linguagem SQL ANSI", `DENY` não pode ser contado como padrão. `REVOKE` e `GRANT` são padrão; `DENY` é dialeto.

> ■ **ALERTA**
>
> Essas quatro viagens explicam por que duas questões quase idênticas têm gabaritos diferentes. **Não é contradição da banca — é taxonomia diferente sendo aplicada.** Em prova, copie a convenção do enunciado: se ele nomear "DQL", respeite o DQL; se listar só "DDL e DML", o `SELECT` volta para o DML.

2.3 Ordem importa: "primeiro DDL, depois DML"

Várias questões pedem para **relacionar comandos na ordem pedida** ("quais expressões são **respectivamente** comandos do tipo DDL e DML"). Nesses casos, acertar a família de cada comando não basta — **a posição também é o gabarito**:

- `ALTER TABLE` e `CREATE INDEX` → DDL + DDL → errado.
- `UPDATE` e `DROP INDEX` → DML + DDL → **invertido** → errado.
- `UPDATE` e `INSERT INTO` → DML + DML → errado.
- `ALTER DATABASE` e `UPDATE` → DDL + DML → **única que respeita a ordem** → correta.
- `INSERT` seguido de `CREATE TABLE` → DML + DDL → **a ordem pedida** → correta.

> ■■ **PEGADINHA**
>
> "Respeitivamente" (ou "seguida de") é a palavra mais perigosa do enunciado. Antes de olhar as letras, monte mentalmente a **dupla** (família do 1º, família do 2º). Se o enunciado pediu DDL e depois DML, procure a alternativa que é **(DDL, DML)** — as três irmãs restantes caem sozinhas. Invertendo a ordem do enunciado para DML e depois DDL, a resposta vira `INSERT` + `CREATE TABLE`. **A letra da alternativa muda, a técnica não.**

2.4 Comando inexistente: a armadilha preferida

Muita questão não cobra classificação — cobra **se o comando existe**. Atenção a sintaxes que soam SQL mas não são:

• `MODIFY` **não atualiza dados.** Em SQL o comando para alterar valor é `UPDATE` (DML). Onde existe `MODIFY`, é o Oracle `ALTER TABLE... MODIFY (coluna tipo)`, que altera a **estrutura** (DDL), não o dado (→ **Errado**).

• `SELECT` **sem lista de colunas.** `SELECT FROM tabela` é sintaxe inválida: ou você nomeia as colunas, ou usa `*`.

• `EXTRACT ALL COLUMN` não é palavra-chave de SQL alguma.

• `DELETE TABLE` não existe — é `DROP TABLE` (item "onde eu erraria" abaixo).

• `ON DELETE CASCADE` não é *instrução* DDL: é **ação referencial** declarada dentro de uma restrição de integridade referencial, não um comando autônomo.

• `LEFT INNER JOIN` não existe: junção só interna (`INNER`) ou só externa (`LEFT`/`RIGHT`/`FULL`). "LEFT INNER" mistura os dois.

• `WHERE` **é cláusula de DML**, nunca de DDL.

• `SELECT * INTO` é outra coisa (cria tabela a partir de consulta); para inserir linha, é `INSERT INTO`.

■ DICA

Regex mental de 2 segundos antes de marcar "isso não existe": *o comando está agindo sobre* ***linha****, sobre* ***estrutura****, ou é só uma* ***cláusula*** *dentro de outro comando?* `WHERE`, `ON`, `HAVING` e `CASCADE` são **cláusulas** — nunca contam como "comando da DDL". Esse é o raciocínio que resolve inteira.

2.5 Triggers: DML em tabela, DDL em banco

Questão recorrente sobre triggers no PL/SQL do Oracle. O que precisa estar claro:

• **Trigger de tabela (ou de visão)** dispara por evento **DML**: `INSERT`, `UPDATE`, `DELETE`. É o caso comum, e pode ser condicional com cláusula `WHEN` avaliada a cada linha afetada.

• **Trigger de DDL existe no Oracle**, mas é de **escopo de banco/esquema** (`BEFORE`/`AFTER DDL ON DATABASE` ou `ON SCHEMA`), não de tabela. Por isso a afirmação "se o trigger for criado em uma tabela ou visão, o evento será DDL" é **falsa** — o escopo já contradiz.

• O termo **"trigger de transição cruzada"** não existe na terminologia do Oracle.

• **Prioridade entre instruções dentro de trigger** não existe por padrão: o controle de concorrência é do SGBD.

■ CONEXAO

Aqui está a ponte para **Segurança da Informação**: trigger é o parente próximo de *auditoria de logs*. Em ITIL 4 e na LGPD, "quem registrou, quando e o que mudou" é requisito de rastreabilidade — e é exatamente o que o par `OLD`/`NEW` entrega. Guarde o par para a questão de trigger e reaproveite na disciplina que pedir rastreabilidade de alterações.

DROP x TRUNCATE x DELETE — o trio que a FGV roteia

Comando	O que remove	Estrutura?	Log?
	DROP	A tabela **como objeto** (estrutura + dados + índices)	■ some

	TRUNCATE	**Todos os registros**, mantém estrutura	Continua
	DELETE	**Registros selecionados**, mantém estrutura	Continua

e são a mesma pergunta com roupas diferentes: *"remover definitivamente a tabela, 'inclusive a estrutura'"* → **DROP TABLE**.

> ■■ **PEGADINHA**
>
> A palavra-chave **"inclusive sua estrutura"** separa DROP de TRUNCATE/DELETE. "Apagar tudo mas manter a tabela" → TRUNCATE. O `DELETE` lê linha por linha e grava no log de auditoria — associe **DELETE ao registro**, **DROP ao objeto**.

3.1 Constraints — as regras declarativas de integridade

Constraint é a regra declarativa que o campo carrega para proteger a integridade dos dados: `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `FOREIGN KEY`, `CHECK`.

- `UNIQUE` + `NOT NULL` → número de processo único e obrigatório.
- `CHECK (status IN ('ABERTO','FECHADO','EM_ANDAMENTO'))` → domínio fechado de valores.
- `AUTO_INCREMENT`/`SERIAL` → gera o ID, mas quem garante a **restrição** é a **constraint**, não a sequence.

> ■■ **PEGADINHA**
>
> A sequence/serial apenas *gera* o número — não garante PRIMARY KEY, UNIQUE, NOT NULL ou CHECK. Pergunta "qual mecanismo garante as regras de integridade?" → **constraints**; "qual gera o próximo valor?" → **sequence**. Pegadinha clássica: olhar o autoincremento e marcar sequence.

3.2 INSERT INTO — a sintaxe certa de inserir

Para **adicionar nova linha** em tabela existente:

```
INSERT INTO tabela (coluna1, coluna2) VALUES (valor1, valor2);
```

armou a cilada com sintaxes "quase SQL": `UPDATE` altera linhas existentes, `ALTER TABLE` muda a estrutura, `SELECT * INTO` e `INSERT NEW` **não são sintaxe válida** de inserção.

Ordem lógica de execução do SELECT

A ordem **lógica** (não é a ordem que você digita):

- ■ `FROM`
- ■ `JOIN / ON`
- ■ `WHERE`
- ■ `GROUP BY`
- ■ `HAVING`
- ■ `SELECT`
- ■ `DISTINCT`

■ `ORDER BY`

■ `LIMIT / OFFSET`

■ DICA

Pergunte-se sempre: **já agrupou?** Se a condição é sobre um **grupo** (média, contagem por departamento), é `HAVING` — o `WHERE` já passou, e o `ORDER BY` só ordena. Condição sobre **linha individual** = `WHERE`.

4.1 Ordenar × limitar — o trio que parece a mesma coisa

Cláusula	Faz o quê	É cega pra ordem?
	ORDER BY	**Ordena** o resultado (maior → menor, etc.)
	LIMIT	**Corta** a quantidade de linhas retornadas
	TOP	Limita quantidade (SQL Server / outros bancos)
	OFFSET	**Pula** N linhas antes de começar a retornar

■■ PEGADINHA

"Listar começando pelas varas com mais processos" → já tem `GROUP BY` + `COUNT(*)`, falta **ordenar o resultado agregado** → **ORDER BY** `total_processos DESC`. TOP/LIMIT limitam volume, não ordem. E em PostgreSQL, paginar é **LIMIT 20 OFFSET 40** — *não* `START` (START é palavra de transação).

4.2 Joins — quem preserva as linhas de quem

Cada junção "preserva" um conjunto diferente de linhas:

Junção	Preserva
	INNER
	RIGHT
	LEFT
	FULL OUTER
	CROSS

■ MEMORIZAR

Palavra do enunciado = mapa: "listar também os clientes **sem pedidos**" + "tabela da **esquerda** (CLIENTE)" → **LEFT OUTER JOIN**. A pegadinha é trocar por RIGHT ou FULL quando o enunciado fixar o lado da esquerda.

Agregações e contagem

- `COUNT(*)` → conta **linhas** que atendem o critério.
- `SUM` → soma **valores numéricos** de coluna (não conta registro!).
- Padrão do servidor em 2+ órgãos: `GROUP BY ID_SERVIDOR HAVING COUNT(DISTINCT ID_ORGAO) >= 2`.

DISTINCT × UNIQUE — a dupla que a FGV troca

- **DISTINCT** → cláusula do **SELECT**: remove duplicatas **do resultado exibido**.
- **UNIQUE** → **constraint de esquema**: impede dados duplicados **de entrarem** na tabela (ex.: dois CPFs iguais).
- `KEY` (MySQL) é sinônimo de **INDEX** (DDL); `NULL` é ausência de valor; `DEFAULT` é valor padrão de preenchimento — todos distratores.

■■ PEGADINHA

DISTINCT **tira** duplicata na hora de exibir; UNIQUE **bloqueia** na porta de entrada. Banca troca os papéis: "cláusula para consulta retornar apenas valores diferentes" → **DISTINCT**.

■ RESUMO

Contagem pede **COUNT + WHERE**; agrupamento filtrado pede **GROUP BY + HAVING**. `COUNT(DISTINCT...)` = "quantas **coisas diferentes** dentro do grupo".

Gatilhos: OLD e NEW

As variáveis de transição do trigger:

- **INSERT** → só `NEW` (o valor novo que entra).
- **DELETE** → só `OLD` (o registro antigo que sai — não existe "exclusão nova").
- **UPDATE** → `OLD` **e** `NEW` juntos (o antes e o depois).

■■ PEGADINHA

Pergunta certeira da banca: "qual variável no DELETE?" → **OLD**. A armadilha é achar que NEW também existe — a linha **morre**, não gera versão nova.

■ ALERTA

E o evento que dispara o trigger? Trigger **de tabela/visao** responde a evento **DML** (`INSERT`/`UPDATE`/`DELETE`). Trigger de **DDL** existe no Oracle, mas é cadastrado no escopo de **banco ou esquema** (`ON DATABASE`/`ON SCHEMA`) — logo, a frase "trigger criado em tabela ou visão com evento DDL" é falsa por escopo, e "trigger de transição cruzada" não é nomenclatura Oracle. Detalhes em 2.5.

View, Procedure e Trigger — o trio de objetos

Objeto	O que é	Padrão
	View	"Tabela virtual" — select salvo, sem copiar dados
	Procedure	Bloco de comandos reutilizável, aceita parâmetros

	Trigger	Executa **sozinha** diante de evento DML

■ MEMORIZAR

VEJO → **FAÇO** → **ACONTECE**: **V**iew (consulto) → **P**rocedure (chamo) → **T**rigger (acontece sozinho). é a questão-mãe desse trio.

Índices e o otimizador (performance)

• **Índice composto B-Tree** segue a **regra do prefixo à esquerda** (*leftmost prefix*): a consulta só aproveita se filtrar pela **primeira coluna** (ou sequência começando nela). Não é obrigatório citar todas as colunas.

• **Índice não é garantia**: o otimizador escolhe por **custo**; estatísticas **desatualizadas** fazem ele "achar" que o índice não compensa e cair num *table scan* — mesmo com índice adequado.

• **EXPLAIN PLAN** mostra o **caminho de acesso** escolhido — é o recurso para diagnosticar *full table scan* causado por **conversões implícitas de tipos**. Não serve para auditoria, criptografia, backup ou reordenação física.

• **Densidade de página**: página quase vazia = **muita leitura de disco** para a mesma quantidade de linhas → mais I/O e mais pressão no Buffer Pool. Densidade baixa é convite ao *table scan*.

• **Page Split só existe em rowstore** (B-tree): quando a página enche, ela se divide. Índice **Columnstore** não sofre *page split* — os dados novos vão para a **Delta Store** e depois são **comprimidos em Rowgroups**.

• **Fragmentação muda o plano de execução**: com índice fragmentado e densidade baixa, o otimizador **prefere o table scan** por custo — um índice esburacado vira "o melhor a fazer é varrer tudo". Índices **pouco fragmentados são excelentes**; não deixe a banca inverter a semântica.

■■ PEGADINHA

Conversão implícita (coluna numérica comparada com string) **invalida o uso de índice** → full scan. Quando a opção mencionar "conversão implícita + índice", é plano de execução.

Particionamento Físico de VLDB (*Partition Pruning*)

Bancos de Dados Muito Grandes (*Very Large Data Bases* - VLDB) utilizam particionamento físico horizontal para viabilizar manutenção e alto desempenho:

• **Partition Pruning (Poda de Partições):** É a técnica essencial de otimização em que o SGBD, ao analisar os predicados da cláusula `WHERE`, **ignora fisicamente** as partições que comprovadamente não contêm dados da consulta. Exemplo: se a consulta filtra vendas de `2024`, partições de `2022` e `2023` não sofrem leitura de disco.

• **Manutenção Granular:** Operações como backup, reconstrução de índices (*rebuild*) e descarte de dados velhos (`DROP PARTITION`) podem ser feitas **no nível da partição individual**, sem exigir bloqueio ou manutenção da tabela inteira.

• **Paralelismo no Oracle:** A execução paralela opera intensamente em consultas (`SELECT` / DQL), comandos DML em lote e instruções DDL — desconfie de alternativas que afirmem ser "exclusiva de DDL".

8.1 A Tríade da Exclusão no SQL

Comando	Subconjunto	O que faz	Log e Transação	Cláusula WHERE?
	DROP TABLE	**DDL**	Dinamita o prédio inteiro: apaga a estrutura do dicionário de dados e todos os registros.	DDL (autocommit implícito)
	TRUNCATE TABLE	**DDL**	Esvazia o prédio mantendo a carcaça: desaloca páginas de dados rapidamente. Reseta auto-incremento.	DDL (log mínimo)
	DELETE FROM	**DML**	Retira os móveis cômodo a cômodo: apaga linhas específicas.	DML (log completo por linha, gera rollback)

8.2 Divisão Relacional em SQL ($A \div B$)

A Divisão Relacional responde a perguntas do tipo: *"Quais alunos estão matriculados em **TODAS** as turmas?"* ou *"Quais clientes compraram **TODOS** os produtos do catálogo?"*.

Como o SQL não possui um operador direto `DIVIDE BY`, a divisão relacional é codificada pelo molde do **duplo NOT EXISTS correlacionado** (dupla negação lógica: *"selecione o aluno para o qual NÃO EXISTE turma em que NÃO EXISTA matrícula"*):

```
SELECT A.Nome
FROM Aluno A
WHERE NOT EXISTS (
SELECT T.TurmaID
FROM Turma T
WHERE NOT EXISTS (
SELECT M.TurmaID
FROM Matricula M
WHERE M.AlunoID = A.AlunoID AND M.TurmaID = T.TurmaID
)
);
```

■ MEMORIZAR

Molde da Divisão: Encontrou `NOT EXISTS` aninhado com outro `NOT EXISTS` correlacionado testando duas entidades vinculadas? O gabarito é **Divisão Relacional** (o teste de quem atendeu a "TODOS" os itens da outra tabela).

8.3 Restrições de Tabela e Valores Padrão

• **PRIMARY KEY = UNIQUE + NOT NULL:** Todo SGBD relacional impõe que uma chave primária rejeite valores repetidos e rejeite nulos simultaneamente.

• **DEFAULT:** Define a "rede de segurança" da inserção. Se o comando `INSERT` omitir o campo, o SGBD aplica automaticamente o valor padrão especificado. Isso **não impede** alterações posteriores via `UPDATE`.

Onde eu erraria e por quê?

Eu marcaria "DELETE TABLE" com cara de comando válido — mas **não existe**; a remoção de objeto é `DROP TABLE`. E em questão de índice composto eu diria "o otimizador reordena os predicados e tanto faz a ordem" — **falso**: a ordem das colunas é fixa e vital, por causa do prefixo à esquerda.

Nos itens novos, meus três candidatos naturais de erro são:

■ `MODIFY` **como sinônimo de** `UPDATE`**.** Parece a palavra que o português sugere e não existe para atualizar dado.

■ `COMMIT` **puxado para DML** só porque "é um comando SQL como outro qualquer".

■ `WHERE`**/**`ON DELETE CASCADE` **promovidos a "comando da DDL"** só porque aparecem dentro de um `CREATE TABLE` — são **cláusulas**, não instruções.

■ **Sequência de** `CREATE TABLE` **/** `INSERT` **/** `GRANT` **fora de ordem**: o instinto acerta as três famílias e mesmo assim erra a alternativa, porque a posição não bate.

■ `ALTER TABLE` **que "afrouxa" dado existente**: penso só na família (DDL, certa) e ignoro que a alternativa exige **adicionar** restrição sobre coluna com repetidos.

O que revisar e quando?

• **A matriz comando → família (2.1)** — é o item de maior retorno em provas de SQL, respondendo imediatamente qualquer questão de classificação de subconjuntos. Revisar na véspera de prova.

• **As quatro viagens da banca (2.2)** — `SELECT` DML×DQL, `COMMIT` fora da DML, `TRUNCATE` como DDL, `DENY` como dialeto. São 4 frases para decorar.

• **"Respectivamente" (2.3)** — técnica de leitura, não conteúdo. Vale treinar em qualquer lista de comandos.

• **Comandos inexistentes (2.4)** — `MODIFY`, `EXTRACT ALL COLUMN`, `SELECT FROM`, `DELETE TABLE`, `LEFT INNER JOIN`. Memorizar a lista curta; é rendimento alto.

• **Mapa dos subgrupos (DDL/DML/DCL/DTL/DQL)** — cobrança recorrente em nível "classifique os comandos".

• **DROP × TRUNCATE × DELETE** — praticamente anual.

• **HAVING/WHERE + agregações** — forte tendência FGV 2026.

• **Triggers: escopo × evento** (2.5 e seção 6) — Oracle cai bastante.

• **Índices/otimizador** — a novidade da FGV 2026/2027; vale revisão antes de simulado.

Lição para outras matérias

A lógica de **subgrupos com responsabilidades separadas** (definição × manipulação × controle) é a mesma do **ITIL 4**: cada prática tem seu "quem define" e "quem executa". E a lição do otimizador — *ter o recurso (índice) ≠ ser obrigado a usá-lo* — é o mesmo "possuir não é utilizar" que a FGV adora em **Governança** (prontidão tecnológica × adoção efetiva).

E há uma lição de **técnica de prova** que vale para qualquer disciplina: as 25 questões de classificação não eram 25 conteúdos diferentes, eram **um** conteúdo com 25 disfarces. Quando a banca repete o mesmo formato, o ganho não está em estudar 25 itens — está em achar o **molde**. Do mesmo jeito que "integridade referencial × integridade de domínio" (tema T04) reaparece com palavras novas, aqui o molde é sempre *estrutura* × *conteúdo* × *permissão* × *transação*. Treine o molde e você mata a família inteira.

Banco de Dados T10 — Propriedades ACID das Transações

> ■■ **INFO**
>
> **Gerência de Transações e Propriedades ACID:** estudo prático de Atomicidade (*tudo ou nada*), Consistência (*preservação das regras e restrições de integridade*), Isolamento (*execução concorrente sem interferência*) e Durabilidade (*persistência definitiva após o commit*).

Do que se trata?

Pensa no caixa do mercado: você passa o cartão, o sistema autoriza, o produto sai da prateleira e o débito cai na sua conta. Se a luz cair no meio desses passos, você não quer ficar sem produto e com o dinheiro debitado. Uma **transação** é exatamente esse "pacote de operações que ou acontece inteiro, ou não acontece nada" — e o **ACID** é o contrato de confiança que o SGBD assina para cumprir a promessa.

Letra	Propriedade	O que garante
	A	**Atomicidade**
	C	**Consistência**
	I	**Isolamento**
	D	**Durabilidade**

É a receita doce de banco: **atomicidade, consistência, isolamento e durabilidade** — nessa ordem, sempre.

Por que cai em prova?

Porque as bancas deixaram de cobrar apenas a tradução literal da sigla e passaram a narrar **um caso concreto de falha ou concorrência** pedindo qual propriedade foi violada (como em provas recentes de TCE-PE, NAV Brasil e MPE-ES).

> ■ **DICA**
>
> Palavra do enunciado = propriedade: **"rollback"** ou **"estado parcial"** → atomicidade · **"transações simultâneas"** ou **"leitura suja"** → isolamento · **"regras de integridade / estado válido"** → consistência · **"após o commit, mesmo em caso de falha"** → durabilidade.

Como as partes se conectam?

• **T07-09 (SQL):** `COMMIT` e `ROLLBACK` são comandos **DTL** — são o "botão" que liga e desliga a promessa ACID na prática (a banca já tentou vendê-los como DDL).

• **T02 (Abordagem relacional):** a **integridade transacional** é o quinto tipo de integridade — ela cuida do dado *durante* a transação, enquanto entidade e referencial cuidam do dado *em repouso*.

• **T10 (ANSI/SPARC):** os três níveis de visão explicam *como* o esquema é apresentado; o ACID explica *como* o dado se comporta quando alguém escreve. São camadas diferentes — não misture.

• **Redes/Infra:** descreve falha entre **dois servidores regionais** — aí entra transação distribuída, o ACID "escalado", que a banca só costuma encostar no enunciado.

Pegadinhas clássicas

■■ PEGADINHA

Atomicidade × durabilidade: a primeira evita **estado parcial antes do commit**; a segunda garante o resultado **depois do commit**. As duas falam de "falha", e a banca conta com você não perceber o **recorte temporal**.

Persistência: é o senso comum do "ficou guardado", mas **não é letra do ACID** — o nome técnico exigido é **durabilidade**.

Disponibilidade: palavra de IA/DI que soa de sistema, mas **não integra o ACID**. Distrator fora do conjunto.

Isolamento × atomicidade: uma não é o "oposto" da outra no sentido de anular — são funções distintas: atomicidade é para transação **sozinha** (tudo ou nada), isolamento é para transação **no meio das outras** (cada uma no seu espaço).

O que separa quem acerta de quem erra?

Quem acerta **localiza o momento do problema** antes de ler as alternativas:

Pergunta de bolso	Resposta
	Falha **no meio**, sobrou metade gravada?
	Uma transação **leu valor temporário** de outra em execução?
	Confirmou e **some depois da queda** do servidor?
	Fim de transação **violou regra de integridade**?

■ ALERTA

é a questão-mãe dessa tabela: I = atomicidade, II = isolamento, III = durabilidade, IV = consistência → **A**. Se você marcar pela primeira palavra que achar "forte" no enunciado, erra — o enunciado de I e III fala de falha nos dois casos, e elas ficam em lados opostos da linha do commit.

Onde eu erraria e por quê?

Eu marcaria **durabilidade** na situação I só porque aparece "falha de energia". É tentador: energia cortada grita "dado perdido, durabilidade!". Mas leia o enunciado: a falha foi **antes da conclusão**, e o estrago é *metade gravada, metade não* — isso é atomicidade. A durabilidade só entra quando a transação **já foi confirmada** e ainda assim se perde (situação III). A mesma pegadinha aparece em, onde o gatilho textual é "totalidade **ou**

completamente desfeita".

Como memorizar sem decorar?

Imagine a transferência **João** → **Maria: R$ 100** (tira R$ 100 de João e põe R$ 100 em Maria):

> ■ **MEMORIZAR**
>
> **A — Atomicidade:** não pode tirar de João e esquecer de pôr em Maria. **Tudo ou nada.**
>
> **C — Consistência:** depois da operação, o banco continua obedecendo às suas regras. **Continua correto.**
>
> **I — Isolamento:** se várias transferências correm juntas, nenhuma enxerga o meio da outra. **Cada transação no seu espaço.**
>
> **D — Durabilidade:** fez COMMIT? Ficou registrado, mesmo que o sistema caia. **Gravou = ficou.**
>
> Frase de fita: **ACID = Tudo ou nada + Correto + Separado + Permanente.**

O que revisar e quando?

- antes de qualquer simulado — são as três formulações que a FGV usou em 2026 (sequência de propriedades, definição isolada e "após o commit").
- Revisar **junto com T07-09**, porque a banca costuma amarrar ACID aos comandos `COMMIT`/`ROLLBACK` (DTL).
- Se aparecer **transação distribuída / dois servidores**, o núcleo cobrado continua sendo o ACID clássico — não entre em pânico procurando protocolos de consenso.

Lição para outras matérias

> ■ **CONEXAO**
>
> O padrão "ler o **recorte temporal** antes de responder" é o mesmo de **Legislação** (qual artigo/parágrafo vale na data dos fatos) e de **Português** (qual tempo verbal o enunciado pede). E o ACID é a versão técnica de qualquer governança: **promessa só vale se houver mecanismo que a garanta** — atomicidade é política de tudo-ou-nada, durabilidade é logging, isolamento é segregação de funções (COBIT/ITIL amam esse paralelo).

Banco de Dados T10 — SGBD: Arquitetura ANSI/SPARC

Três Níveis de Arquitetura

Nível	Visão
	Externo

	Conceitual
	Interno

Independência de Dados

Independência de dados — capacidade de modificar a definição de um esquema de um nível **sem afetar** a definição de esquema de um nível **mais alto**.

Tipo	Escopo
	Independência física de dados
	Independência lógica de dados

> ■ **Dica para questões**
>
> Alteração no nível interno sem impacto no conceitual → **independência física**.
>
> Alteração no esquema conceitual/lógico sem impacto nas visões externas → **independência lógica**.

Banco de Dados T11 — Banco de dados NoSQL

> ■■ **INFO**
>
> **Bancos de Dados NoSQL:** características arquiteturais (*schema-free*, escalabilidade horizontal, consistência eventual / BASE), comparação com SGBDs relacionais e estudo das quatro famílias principais (Chave-Valor, Documentos, Colunar e Grafos).

Do que se trata?

Pensa na despensa relacional: tudo com etiqueta, prateleira com medida, nada fora do lugar — ótimo para não perder nada, péssimo quando a família dobra e a despensa não cabe mais na parede. O **NoSQL** nasceu dessa dor: bancos que **afrouxam as regras do modelo relacional** para conseguir crescer em largura (muitas máquinas, não uma máquina mais gorda) sem drama de migração de esquema.

As características que costumam cair na prova:

Característica	O que significa no dia a dia
	Escalabilidade horizontal
	Schema-free / esquema flexível
	Suporte nativo a replicação
	API simples
	Consistência eventual

O tópico 11 do edital andava sem material aqui justamente porque a cobrança era tímida — e ela mudou de patamar com a FGV 2026.

Por que cai em prova?

Porque o edital colocou **"Banco de dados NoSQL" como tópico próprio** e a FGV começou a cobrar do jeito que gosta: **comparação com o relacional**. Em 2026 (AL-RO) ela descreveu uma aplicação de cache de sessões em NoSQL Chave-Valor e pediu **a principal vantagem frente a um SGBD relacional tradicional** — ou seja, não pediu definição, pediu *escolha do modelo certo para o caso*.

> ■ **DICA**
>
> Palavra-chave do enunciado = modelo: **cache, sessões, carrinhos, tokens, latência mínima, "busca pela chave"** → **chave-valor**. **Documento JSON complexo** → documental. **Relação entre entidades / rede social** → grafo. **Foreign keys, ACID, joins ricos** → relacional (está fora de NoSQL — é o concorrente da questão).

Como as partes se conectam?

• **T06 — Modelagem dimensional:** a desnormalização já citada lá tem sua versão radical no NoSQL: o registro vira **um bloco único**, sem join interno nem externo. Mesma lógica, grau maior.

• **T10 — Propriedades ACID:** a **consistência eventual** é o contrato oposto ao "commit já vale em toda parte". São duas promessas diferentes de coerência — confundir as duas é confundir idiomas.

• **T02 — Abordagem relacional:** no NoSQL, a **integridade referencial** não é garantida por FK — quem zela pela coerência entre registros é a **aplicação**. Trocaram o guarda do banco pelo guarda da porta.

• **Infra / Redes:** escalabilidade horizontal e replicação nativa são os mesmos temas de **cluster, alta disponibilidade e nuvem** (Suporte e Infraestrutura) — o NoSQL só empurra essa discussão para dentro do modelo de dados.

As pegadinhas clássicas

é a questão-mãe: cada distrator descreve **outro modelo** (ou o concorrente relacional).

Alternativa	Descreve na verdade	Por que é cilada
	A — esquemas flexíveis + JSON complexo	**Documental**
	B — FK + transações ACID completas	**Relacional**
	C — escalabilidade horizontal + latência de milissegundos + busca pelo endereço do valor	**Chave-valor**
	D — linguagem declarativa rica + JOINS	**Relacional**
	E — redes sociais, foco na relação entre entidades	**Grafo**

> ■■ **PEGADINHA**
>
> **Schema-free não é sinônimo de chave-valor** — é quase sinônimo de *NoSQL inteiro* (e a que usa esse argumento é justamente a do modelo **documental**). A pegadinha da FGV é misturar os tipos de NoSQL entre si e ainda importar vantagens do relacional para dentro deles. Leia o que a **aplicação** faz, não o que a alternativa tem de "bonito em geral".

O que separa quem acerta de quem erra?

Quem acerta **descreve a aplicação antes de olhar as alternativas** — é a pergunta "o que esse sistema faz o dia inteiro?" que aponta o modelo:

Modelo	Forma dos dados	Feito para...	Sinal no enunciado
	Chave-valor	par `chave` → `valor`	cache, sessão, carrinho, token
	Documental	documento JSON/XML inteiro	conteúdo com formato variável
	Grafo	nós + arestas	relações e caminhos
	Colunar	colunas agrupadas por família	análise de volume em massa
	Relacional (o concorrente)	tabelas + JOIN	integridade e consulta complexa

A chave-valor não faz JOIN nem consulta complexa — por isso oferece **altíssimo desempenho, baixa latência e grande escalabilidade horizontal**: quando a aplicação conhece a chave, a busca é **direta**, sem varredura. Baixíssima latência (em milissegundos ou menos), alto desempenho de leitura e escrita e fácil crescimento por novos servidores.

Onde eu erraria e por quê?

Eu marcaria a letra **A** — "esquemas flexíveis e JSON" é a frase mais NoSQL que existe, e schema-free está na lista de características do tópico. Só que a pergunta é **principal vantagem do chave-valor frente ao relacional**, e esquema flexível é o forte do modelo **documental**. E na letra **B** eu hesitaria: ACID, FK e integridade soam "corretos demais" para serem distratores — mas é exatamente o pacote do **relacional tradicional**, o outro lado da comparação. Erramos quando respondemos pela *qualidade* da alternativa em vez da *pertencência* ao modelo cobrado.

Como memorizar sem decorar?

> ■ **MEMORIZAR**
>
> Características do NoSQL numa frase de bolso: **"Sem esquema, sem join, com chave, com réplica e com paciência."** — paciência = **consistência eventual** (espera as cópias convergirem). E chave-valor = **armário de gavetas numeradas**: você chega com o número da gaveta, abre e pega — não existe "procurar por cor nas outras gavetas". Quem tem a chave, tem o valor.

O que revisar e quando?

• Revise a comparação entre o modelo Chave-Valor e o SGBD relacional antes dos simulados; o formato de enunciado aplicado comparando escalabilidade e esquema com o modelo relacional é o mais recorrente.

• Releia **T10 (ACID)** junto: consistência eventual × atomicidade/durabilidade é o par que a banca troca com mais frequência.

• Releia **T06 (desnormalização)**: o argumento "registro em bloco único, sem join" é o mesmo, com outro nome.

• Os bancos NoSQL formam a base das arquiteturas modernas de **bancos em memória (Redis)**, **Data Lakes** e **Big Data**.

Lição para outras matérias

■ **CONEXAO**

Três reciclagens: **escalabilidade horizontal e replicação** = mesmo tema de **Infraestrutura** (cluster, alta disponibilidade, nuvem) e de **Redes** (distribuir carga em vez de centralizar); **consistência eventual** = a ideia de que *redes convergem* — dá tempo, mas não é instantâneo; **schema-free** = cuidado com **Governança de TI**: empurrar a regra para a aplicação não elimina a regra, só tira do catálogo — quem não tem esquema no banco precisa ter metadados na cabeça da equipe, senão vira *data swamp* (T01, Introdução).
